

Lecture 17 - Part 1: FLDA for Multiples Classes

Last class, we started the discussion of **linear discriminant functions for classification**. A linear discriminant function for classification as the following mapper function form:

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$$

where $\mathbf{w}$ are the parameters of the model ($D \times 1$ vector), w_0 the intercept/bias of the model (scalar value), and $y(\mathbf{x})$ is the output value.

This mapper function resembles a mapper function for a **regression** task. In fact, without any other transformation, the output value $y(\mathbf{x})$ is continuous-valued.

Take the example of **Fisher's Linear Discriminant Analysis (FLDA)** covered last class, where a linear discriminant function is found such that:

1. the between-class scatter, S_B , is maximized (separated)
2. the within-class scatter, S_W , is minimized (compact)

That is,

$$\arg_{\mathbf{w}} \max J(\mathbf{w}) = \arg_{\mathbf{w}} \max \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

For a 2-class problem, we use the following **decision rule**:

$$\text{Decide } \begin{cases} \mathbf{x} \in C_1 & \text{if } y(\mathbf{x}) > 0 \\ \mathbf{x} \in C_2 & \text{if } y(\mathbf{x}) < 0 \end{cases}$$

For a multi-class problem (more than 2), a one-vs-all approach can lead to ambiguous regions. Thus, we can first define k discriminant functions, $y_k(\mathbf{x})$, then the decision rule is

$$\text{Decide } \mathbf{x} \in C_i \text{ if } y_i(\mathbf{x}) > y_j(\mathbf{x}), \forall i \neq j$$

LDA Discriminant Function

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

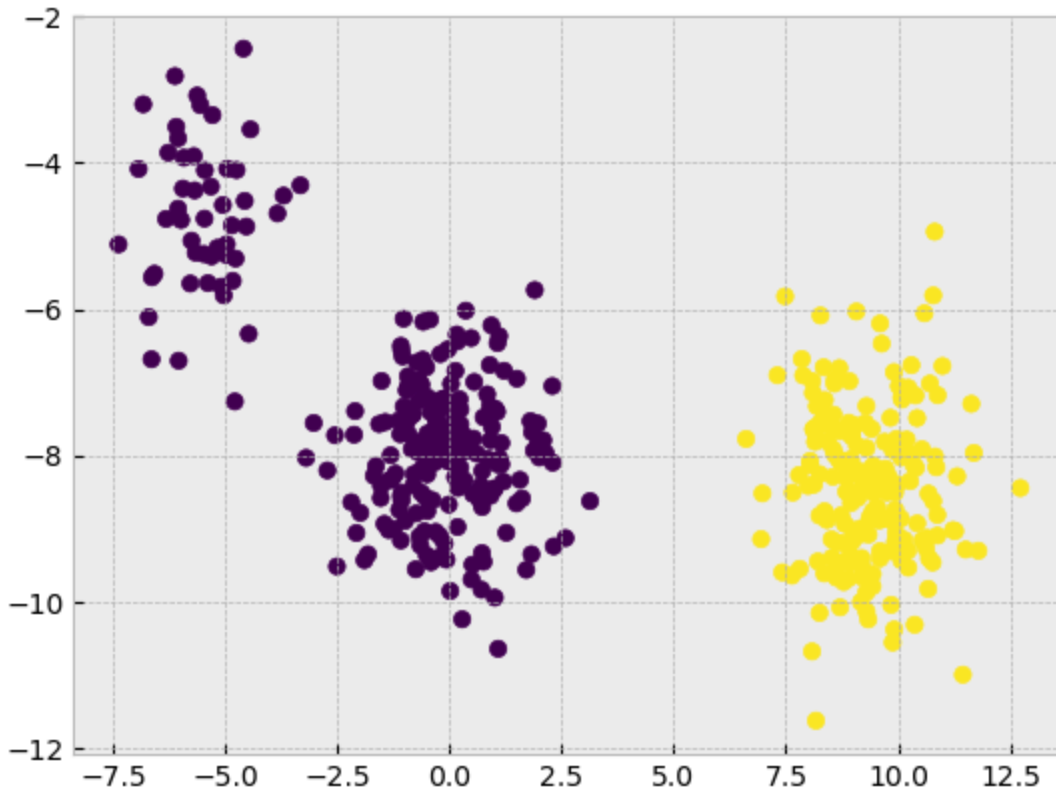
```
%matplotlib inline
plt.style.use('bmh')
from sklearn.preprocessing import StandardScaler

X = np.load('data.npy')
t = np.load('labels.npy')

plt.scatter(X[:,0], X[:,1], c=t);

X.shape, t.shape
```

Out[1]: ((450, 2), (450,))



```
In [2]: # Class means
m1 = np.mean(X[t==0,:],axis=0).reshape(-1,1)
m2 = np.mean(X[t==1,:],axis=0).reshape(-1,1)

# Between-class scatter matrix
SB = (m2-m1)@(m2-m1).T

SB
```

Out[2]: array([[110.47181268, -11.5548902],
[-11.5548902 , 1.20859325]])

```
In [3]: # within-class scatter
SW = (X[t==0,:]-m1.T).T@(X[t==0,:]-m1.T) + (X[t==1,:]-m2.T).T@(X[t==1,:]-m2.T)

SW
```

```
Out[3]: array([[1663.56755767, -720.87195412],
               [-720.87195412,  869.21605526]])
```

```
In [4]: # Eigendecomposition of  $SW^{-1}@SB$ 

# eigendecomposition for hermitian matrices
L, E = np.linalg.eig(np.linalg.inv(SW)@SB)
L, E
```

```
Out[4]: (array([8.78456843e-02, 2.79829245e-18]),
         array([[0.82349848, 0.10402832],
               [0.56731847, 0.99457434]]))
```

```
In [5]: # the w vector is the eigenvector with largest eigenvalue

w = E[:,0]
w
```

```
Out[5]: array([0.82349848, 0.56731847])
```

```
In [6]: # Verify that it is a length=1 vector

np.linalg.norm(w)
```

```
Out[6]: 1.0
```

```
In [7]: # Intercept/bias term

w0 = (m1+m2).T@w/2
w0
```

```
Out[7]: array([-1.11141939])
```

```
In [8]: def discriminant(data, labels, w, w0):
# perpendicular vector to w (swap entries and negate one of them)
w_perp = np.array([w[1], -w[0]])

# points on the real line for plotting
x = np.linspace(-15,15,data.shape[0])

# plotting direction of projection
proj_line = x * w.reshape(-1,1)

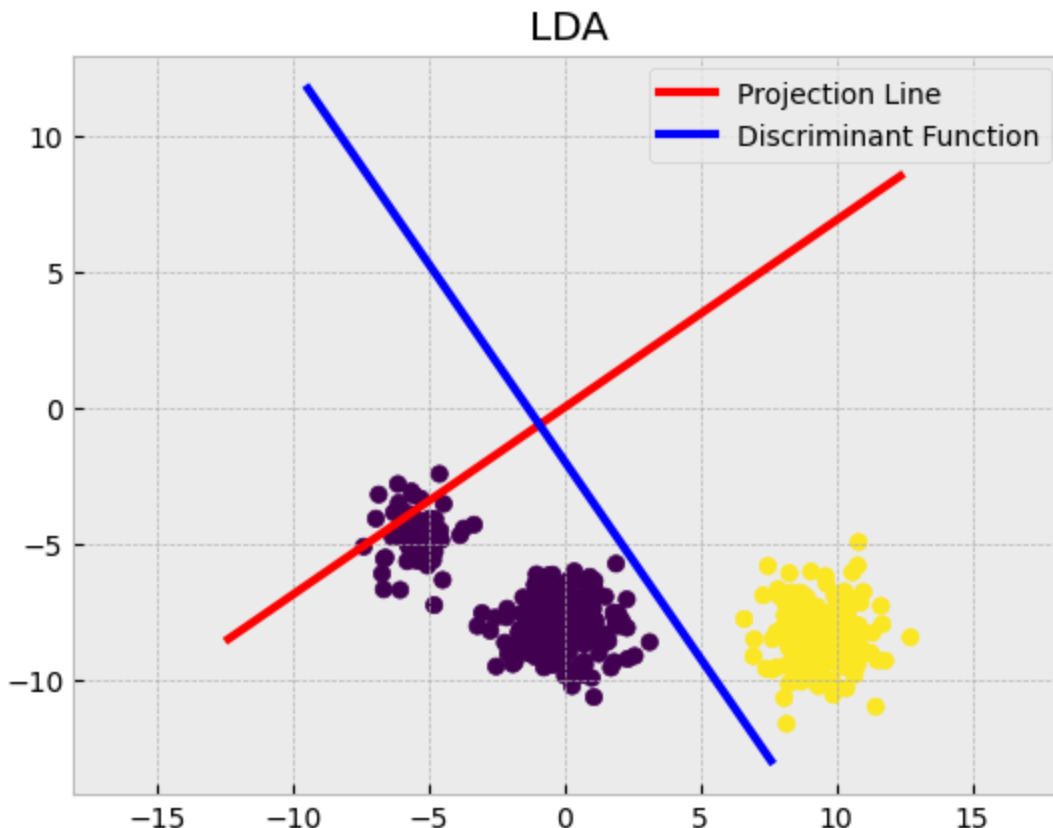
# discriminant function (orthogonal line of direction of projection)
discriminant_fct = w0 * w.reshape(-1,1) + x * w_perp.reshape(-1,1)

return proj_line, discriminant_fct

proj_line, discriminant_fct = discriminant(X, t, w, w0);

plt.scatter(X[:,0],X[:,1],c=t)
plt.plot(proj_line[0], proj_line[1], 'red', linewidth=3,
         label='Projection Line')
plt.plot(discriminant_fct[0,:], discriminant_fct[1,:], 'blue', linewidth=3,
```

```
label='Discriminant Function')
plt.title('LDA'); plt.axis('equal'); plt.legend(loc='best', fontsize=10);
```



Least Squares Discriminant Function

```
In [9]: X_mat = np.hstack((np.ones((X.shape[0],1)), X))
```

```
X_mat.shape
```

```
Out[9]: (450, 3)
```

```
In [10]: w_ls = np.linalg.inv(X_mat.T@X_mat)@X_mat.T@t
```

```
w_ls
```

```
Out[10]: array([0.62035586, 0.09300222, 0.0640704 ])
```

```
In [11]: x = np.linspace(-10,10,X.shape[0])
```

```
y_ls = -w_ls[0]/w_ls[2] - (w_ls[1]/w_ls[2])*x
```

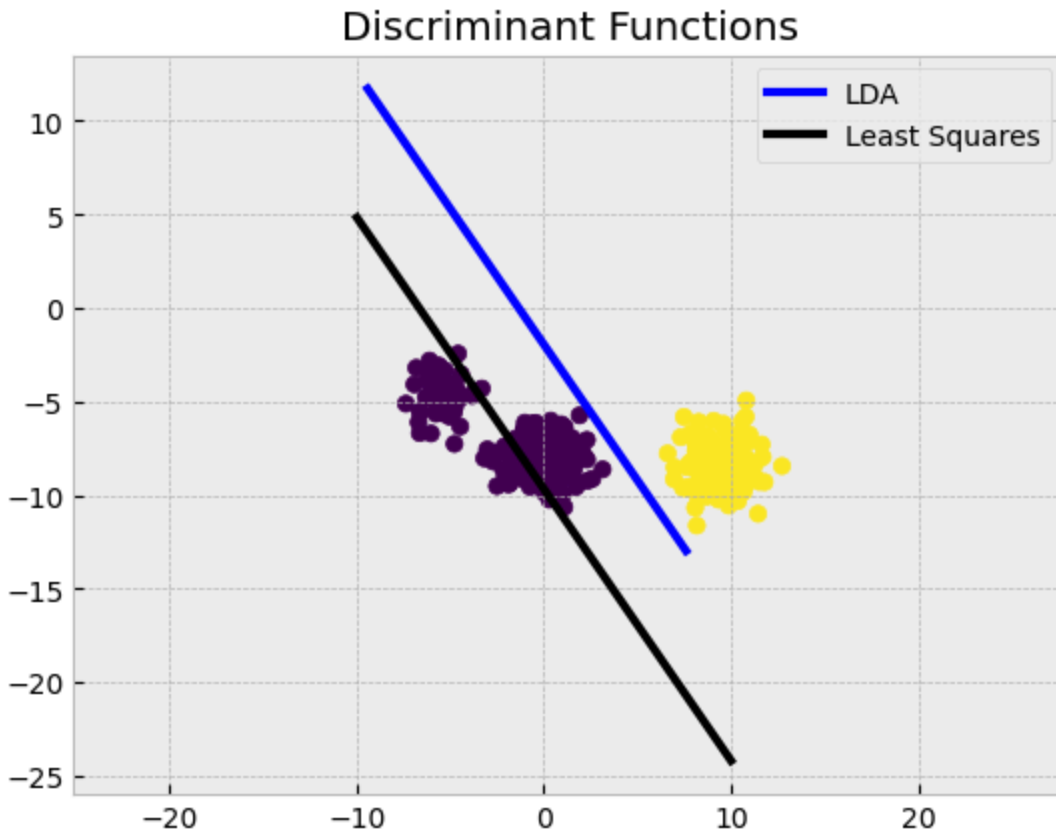
```
y_ls.shape
```

```
Out[11]: (450,)
```

```
In [12]: plt.scatter(X[:,0],X[:,1],c=t)
```

```
plt.plot(discriminant_fct[0,:], discriminant_fct[1,:], 'blue', linewidth=3, label='Discriminant Function')
```

```
plt.plot(x, y_ls, 'black', linewidth=3, label='Least Squares')
plt.title('Discriminant Functions'); plt.axis('equal'); plt.legend(loc='best')
```



Multiple Classes

Considering the extension of linear discriminants to $K > 2$ classes. We might be tempted to build a K -class discriminant by combining a number of two-class discriminant functions. However, this leads to some serious difficulties.

- Consider the use of $K - 1$ classifiers each of which solves a two-class problem of separating points in a particular class C_k from points not in that class. This is known as a **one-versus-all** classifier.
- An alternative is to introduce $K(K - 1)/2$ binary discriminant functions, one for every possible pair of classes. This is known as a **one-versus-one** classifier. Each point is then classified according to a majority vote amongst the discriminant functions. However, this too runs into the problem of ambiguous regions.

```
In [13]: from IPython.display import Image
Image('figures/Figure4.2.png', width=700)
# Source: Bishop textbook
```

Out [13]:

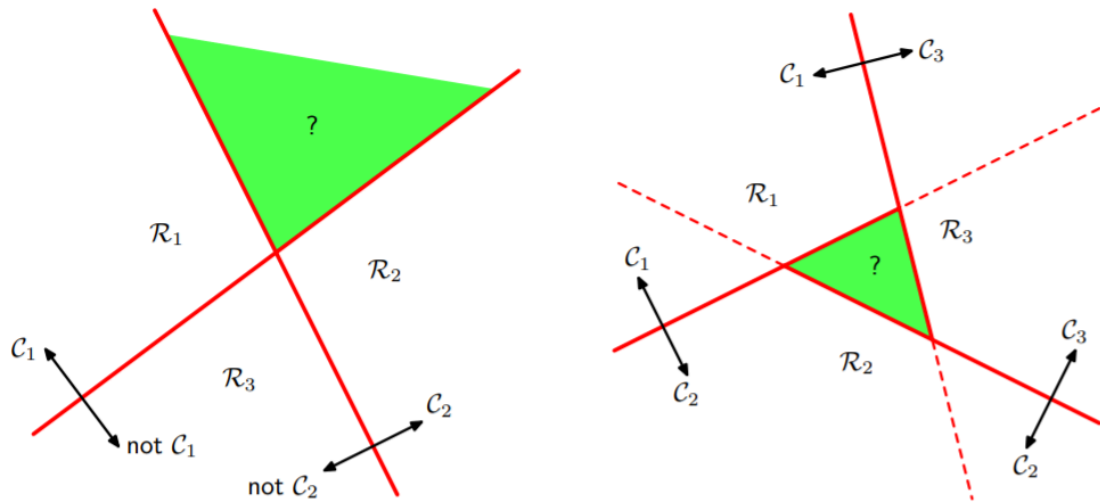


Figure 4.2 Attempting to construct a K class discriminant from a set of two class discriminants leads to ambiguous regions, shown in green. On the left is an example involving the use of two discriminants designed to distinguish points in class C_k from points not in class C_k . On the right is an example involving three discriminant functions each of which is used to separate a pair of classes C_k and C_j .

We can avoid these difficulties by considering a **single K -class discriminant** comprising K linear functions of the form

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k$$

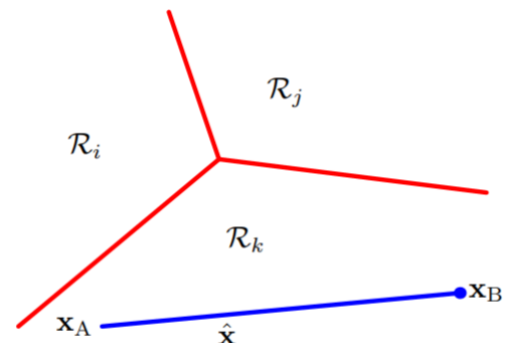
and then assigning a point $\mathbf{x}$ to class C_k if $y_k(\mathbf{x}) > y_j(\mathbf{x})$ for all $j \neq k$. The decision boundary between class C_k and class C_j is therefore given by $y_k(\mathbf{x}) = y_j(\mathbf{x})$ and hence corresponds to a $(D - 1)$ -dimensional hyperplane defined by

$$(\mathbf{w}_k - \mathbf{w}_j)^T \mathbf{x} + (b_k - b_j) = 0$$

This has the same form as the decision boundary for the two-class case.

In [14]: `Image('figures/Figure4.3.png',width=800)`
 # Source: Bishop textbook

Out [14]: **Figure 4.3** Illustration of the decision regions for a multiclass linear discriminant, with the decision boundaries shown in red. If two points $\mathbf{x}_A$ and $\mathbf{x}_B$ both lie inside the same decision region $\mathcal{R}_k$, then any point $\hat{\mathbf{x}}$ that lies on the line connecting these two points must also lie in $\mathcal{R}_k$, and hence the decision region must be singly connected and convex.



The decision regions of such a discriminant are always **singly connected** and **convex**. To see this, consider two points x_A and x_B both of which lie inside decision region R_k . Any point $\hat{\mathbf{x}}$ that lies on the line connecting x_A and x_B can be expressed in the form

$$\hat{\mathbf{x}} = \alpha x_A + (1 - \alpha) x_B$$

where $0 \leq \alpha \leq 1$. From the linearity of the discriminant functions, it follows that

$$y_k(\hat{\mathbf{x}}) = \alpha y_k(x_A) + (1 - \alpha) y_k(x_B)$$

Because both x_A and x_B lie inside R_k , it follows that $y_k(x_A) > y_j(x_A)$, and $y_k(x_B) > y_j(x_B)$, for all $j \neq k$, and hence $y_k(\hat{\mathbf{x}}) > y_j(\hat{\mathbf{x}})$, and so $\hat{\mathbf{x}}$ also lies inside R_k . Thus R_k is singly connected and convex.

Fisher's LDA for Multiples Classes

We now consider the generalization of the Fisher discriminant to $K > 2$ classes, and we shall assume that the dimensionality D of the input space is greater than the number K of classes. Next, we introduce $M > 1$ linear "features" $\mathbf{y}_k = \mathbf{w}_k^T \mathbf{x}$, where $k = 1, \dots, M$ and $M \leq D$.

These feature values can conveniently be grouped together to form a vector $\mathbf{y}$. Similarly, the weight vectors $\{\mathbf{w}_k\}$ can be considered to be the columns of a matrix $\mathbf{W}$, so that

$$\mathbf{y} = \mathbf{W}^T \mathbf{x}$$

Note that again we are not including any bias parameters in the definition of $\mathbf{y}$. The generalization of the **within-class scatter matrix** to the case of K classes is as follows:

$$S_W = \sum_{k=1}^K S_k$$

where

$$S_k = \frac{1}{N_k} \sum_{n \in C_k} (x_n - \vec{\mathbf{m}}_k)^T (x_n - \vec{\mathbf{m}}_k)$$

$$\vec{\mathbf{m}}_k = \frac{1}{N_k} \sum_{n \in C_k} x_n$$

and N_k is the number of patterns in class C_k . In order to find a generalization of the **between-class scatter matrix**, let's consider first the total covariance matrix:

$$S_T = \frac{1}{N} \sum_{n=1}^N (x_n - \vec{\mathbf{m}})(x_n - \vec{\mathbf{m}})^T$$

where $\vec{\mathbf{m}}$ is the mean of the total data set

$$\vec{\mathbf{m}} = \frac{1}{N} \sum_{n=1}^N x_n = \frac{1}{N} \sum_{k=1}^K N_k \vec{\mathbf{m}}_k$$

and $N = \sum_{k=1}^K N_k$ is the total number of data points.

The total covariance matrix can be decomposed into the **sum of the within-class scatter matrix plus an additional matrix** S_B , which we identify as a measure of the **between-class scatter matrix**:

$$S_T = S_W + S_B$$

where

$$S_B = \sum_{k=1}^K N_k (\vec{\mathbf{m}}_k - \vec{\mathbf{m}})(\vec{\mathbf{m}}_k - \vec{\mathbf{m}})^T$$

These covariance matrices have been defined in the original x-input space. We can now define similar matrices in the projected M -dimensional y-space:

$$s_W = \sum_{k=1}^K \sum_{n \in C_k} (\mathbf{y}_n - \mu_k)(\mathbf{y}_n - \mu_k)^T$$

and

$$s_B = \sum_{k=1}^K N_k (\mu_k - \mu)(\mu_k - \mu)^T$$

where

$$\mu_k = \frac{1}{N_k} \sum_{n \in C_k} y_n$$

$$\mu = \frac{1}{N} \sum_{k=1}^K N_k \mu_k$$

Once again, we wish to construct a scalar that is large when the between-class covariance is large and when the within-class covariance is small.

$$J(\mathbf{W}) = \frac{s_B}{s_W}$$

This criterion can then be rewritten as an explicit function of the projection matrix $\mathbf{W}$ in the form

$$J(\mathbf{w}) = \frac{\mathbf{W} S_B \mathbf{W}^T}{\mathbf{W} S_W \mathbf{W}^T}$$

Maximization of such criteria is straightforward, though somewhat involved. The weight values are determined by those eigenvectors of $S_W^{-1} S_B$ that correspond to the M

largest eigenvalues.

There is one important result that is common to all such criteria, which is worth emphasizing. We first note that S_B is composed of the sum of K matrices, each of which is an outer product of two vectors and therefore of rank 1. In addition, only $(K - 1)$ of these matrices are independent. Thus, S_B has rank at most equal to $(K - 1)$ and so there are at most $(K - 1)$ nonzero eigenvalues.

This shows that the projection onto the $(K - 1)$ -dimensional subspace spanned by the eigenvectors of S_B does not alter the value of $J(\mathbf{w})$, and so we are therefore unable to find more than $(K - 1)$ linear "features".

Limitations of FLDA

1. LDA produces at most $K - 1$ feature projections, where K is the number of classes.
2. If the classification error estimates establish that more features are needed, some other method must be employed to provide those additional features.
3. LDA is a parametric method (it assumes unimodal Gaussian likelihoods).
4. If the distributions are significantly non-Gaussian, the LDA projections may not preserve complex structure in the data needed for classification.
5. LDA will also fail if discriminatory information is not in the mean but in the variance of the data.

A popular variant of LDA are the **Multi-Layer Perceptrons** (or MLPs).
